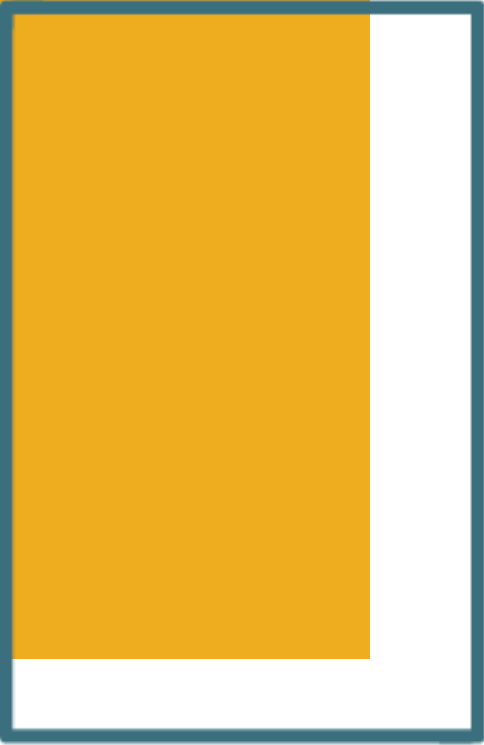




09

Performance Improvement and Metrics



Machine Learning Performance

Enhancing a model's performance can be challenging at times. I'm sure, a lot of you would agree with me if you've found yourself stuck in a similar situation. You try all the strategies and algorithms that you've learned. Yet, you fail at improving the accuracy of your model. Previously we have learned how to improve through ensemble learning.

Adding More Useful Data

Researchers have demonstrated that massive data can lead to lower estimation variance and hence better predictive performance. More data increases the probability that it contains useful information, which is advantageous. We might not get an option to add more data.

For example: we do not get a choice to increase the size of training data in data science competitions. But while working on a company project, you are suggested to ask for more data, if possible.

But please take a note that there comes a stage where even adding infinite amounts of data cannot improve any more accuracy.

"It is not just big data, but good (quality) data which helps us build better performing ML models. If we have a huge data repository with features which are too noisy or not having enough variation to capture critical patterns in the data, any ML models will effectively be useless regardless of the data volume."



Data Cleaning

Data Cleaning means the process of identifying the incorrect, incomplete, inaccurate, irrelevant or missing part of the data and then modifying, replacing or deleting them according to the necessity. Data cleaning is considered a foundational element of basic data science.

Handling Imbalanced Data

Imbalanced data refers to those types of datasets where the target class has an uneven distribution of observations, i.e. one class label has a very high number of observations, and the other has a very low number of observations.

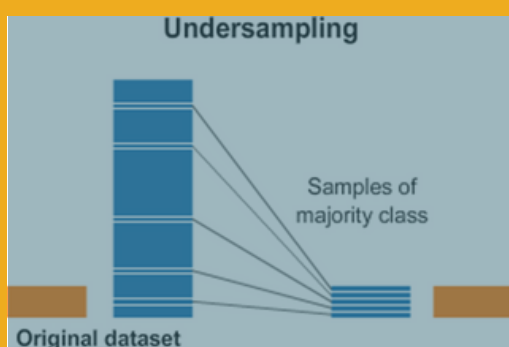
Example: Disease diagnosis, Customer churn prediction, Fraud detection, Natural disaster.

There are some techniques to deal with the imbalanced problems:

- Using the right evaluation metrics
- Resampling (under-sampling, over-sampling)
- Using Ensemble methods

Under-Sampling (Down-Sampling)

It aims to balance class distribution by randomly eliminating majority class examples. This is done until the majority and minority class instances are balanced out.



Advantages

- Run-time can be improved by decreasing the amount of training dataset.
- Helps in solving the memory problems.

Disadvantages

- Losing some critical information
- Sample chosen by random under sampling may be a biased sample.

Over-Sampling (Up-Sampling)

Over-Sampling increases the number of instances in the minority class by randomly replicating them in order to present a higher representation of the minority class in the sample.

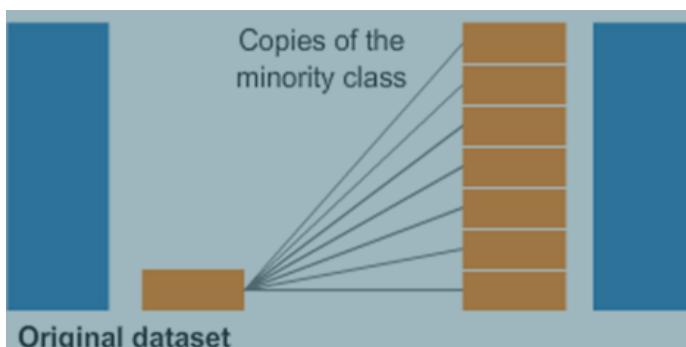
Advantages

- No information loss.
- Outperforms under sampling

Disadvantages

- It increases the likelihood of overfitting since it replicates the minority class events.

Oversampling



This process can be divided into 2 steps:

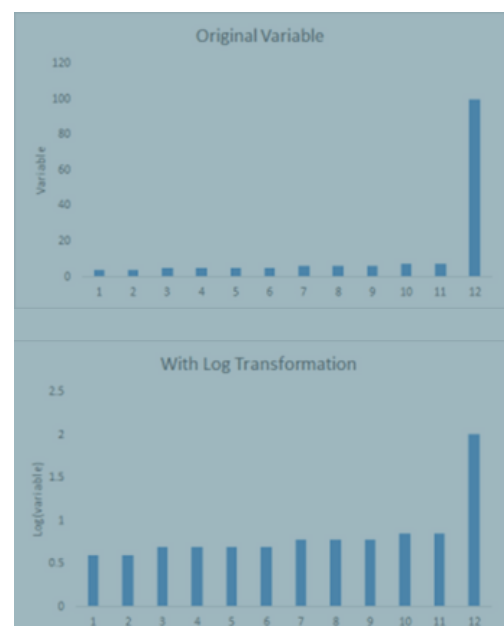
- Feature transformation. To remove skewness, normalize the variables or change the scale.
- Feature Creation. Deriving new variable(s) from existing variables is known as feature creation. It helps to unleash the hidden relationship of a data set.

Let's say, we want to predict the number of transactions in a store based on transaction dates. Here transaction dates may not have direct correlation with the number of transactions, but if we look at the day of a week, it may have a higher correlation. In this case, the information about day of a week is hidden.



Feature Engineering

This step helps to extract more information from existing data. New information is extracted in terms of new features. These features may have a higher ability to explain the variance in the training data.



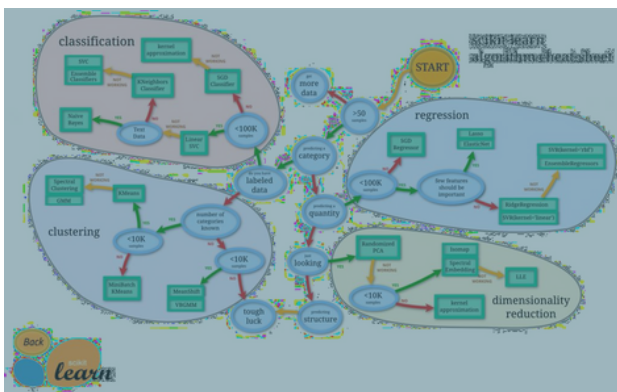
Feature Selection

Feature Selection is a process of finding out the best subset of attributes which better explains the relationship of independent variables with target variables. You can select features based on:

- Domain knowledge
- Visualization
- Statistical parameters.

Compare Multiple Algorithms

Hitting at the right machine learning algorithm is the ideal approach to achieve higher accuracy. But, it is easier said than done.



This intuition comes with experience and incessant practice. Some algorithms are better suited to a particular type of data sets than others. Hence, we should apply all relevant models and check the performance.

Feature Extraction

Feature extraction is for creating a new, smaller set of features that still captures most of the useful information. Again, feature selection keeps a subset of the original features while feature extraction creates new sets of components.

Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA) both reduce the dimensionality of data by looking for linear combinations of the features which best explain the data. LDA focuses on finding a feature subspace that maximizes the separability between the groups. PCA focuses on capturing the direction of maximum variation in the data set.

Algorithm Tuning

We know that machine learning algorithms are driven by parameters. These parameters majorly influence the outcome of the learning process. The objective of parameter tuning is to find the optimum value for each parameter to improve the accuracy of the model. To tune these parameters, you must have a good understanding of these meanings and their individual impact on the model. You can repeat this process with a number of well performing models.

For example: In a random forest, we have various parameters like `max_features`, `number_trees`, `random_state`, `oob_score` and others. Intuitive optimization of these parameter values will result in better and more accurate models.

K-Fold Cross Validation

Cross-validation is a resampling procedure used to evaluate machine learning models on a limited data sample.



The procedure has a single parameter called k that refers to the number of groups that a given data sample is to be split into. As such, the procedure is often called k -fold cross-validation. When a specific value for k is chosen, it may be used in place of k in the reference to the model, such as $k=5$ becoming 5-fold cross-validation.

Performance Metrics in Machine Learning

The model performance gives a measure of how well a Machine Learning model is performing. But, is it all about the accuracy of the model's predictions? Let's see what metrics are used in machine learning other than accuracy.

Please note that metrics are different with loss functions. Loss functions also show a measure of model performance, but they are used to optimize our model while metrics to judge the model performance have nothing to do with optimization. Loss functions might differ for several algorithms, but metrics don't need to differ. Metrics only differ based on the application (classification/regression).

Confusion Matrix

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

Confusion matrix is a table that is often used to describe the performance of a classification model (or "classifier") on a set of test data for which the true values are known.

Measurement	Formula
Accuracy	$\frac{TP + TN}{P + N}$
Misclassification rate	$\frac{FP + FN}{P + N}$
Sensitivity	$\frac{TP}{P}$
Specificity	$\frac{TN}{N}$
Precision	$\frac{TP}{TP + FP}$
Recall	$\frac{TP}{TP + FN}$
F, F ₁ , F-score	$\frac{2 \times \text{precision} \times \text{sensitivity}}{\text{precision} + \text{sensitivity}}$
F _β , where β is real non negative	$\frac{(1 + \beta^2) \times \text{precision} \times \text{sensitivity}}{\beta^2 \times \text{precision} + \text{sensitivity}}$

Regression Metrics

Mean squared error	$MSE = \frac{1}{n} \sum_{t=1}^n e_t^2$
Root mean squared error	$RMSE = \sqrt{\frac{1}{n} \sum_{t=1}^n e_t^2}$
Mean absolute error	$MAE = \frac{1}{n} \sum_{t=1}^n e_t $
Mean absolute percentage error	$MAPE = \frac{100\%}{n} \sum_{t=1}^n \left \frac{e_t}{y_t} \right $

Picture Source

- [pexels.com](https://www.pexels.com)